Trabajo Práctico Integrador: CRUD de Menú de Cafetería

Gestión e Iteración de Arrays de Strings sin Métodos de Objetos

1. Objetivo del Proyecto

El objetivo principal de este trabajo integrador es afianzar el dominio práctico de arreglos (arrays) en JavaScript o TypeScript. Construirás una aplicación orientada a consola para gestionar el menú y stock de productos de una **Cafetería Especializada**, implementando las operaciones esenciales de un CRUD (Create, Read, Update, Delete), funciones callback, filtrados, transformaciones y ordenamiento.

> **Reglas Estrictas del Proyecto**
>
> RESTRICCIÓN CLAVE DEL EJERCICIO:
>
> • NO se permite el uso de Objetos ({}).
>
> • Toda la estructura de datos del menú debe mantenerse únicamente mediante un array unidimensional de cadenas de texto (string[]).
>
> • Puedes resolver el desafío utilizando JavaScript estándar (ES6+) o TypeScript.

2. Array de Referencia (Menú de Cafetería)

Deberás inicializar la aplicación utilizando exactamente el siguiente arreglo base de productos de cafetería. Copia e integra este array en tu archivo principal de código:

https://gist.github.com/pmarino2013/904a9f665d9e2193f03e9d65643b6120

3. Consignas y Puntos a Resolver

► Punto 1: Visualización del Menú (Read / forEach)

Crea una función llamada `mostrarMenu()` que recorra el array utilizando `.forEach()` e imprima en consola cada producto numerado según su índice (ejemplo: `1 - Café Espresso Clásico`).

Pablo Marino

► Punto 2: Códigos de Menú y Precios Estimados (map)

Implementa una función `generarCodigosMenu()` que utilice `.map()` para devolver un nuevo array donde cada ítem posea un código único de producto calculado a partir de un identificador numérico o timestamp más su índice (ejemplo: `CAF-101 - Café Cappuccino Italiano`). Muestra en consola la nueva lista.

► Punto 3: Incorporación de Nuevos Productos (Create / push, unshift, splice)

Desarrolla las siguientes funciones de alta:

a) `agregarAlFinal(item)` utilizando `.push()`.

b) `agregarAlInicio(item)` utilizando `.unshift()`.

c) `insertarEnPosicion(indice, item)` utilizando únicamente `.splice()`.

► Punto 4: Actualización de Elemento del Menú (Update / findIndex & splice)

Implementa la función `actualizarProducto(nombreBuscado, nuevoNombre)` que busque la posición exacta del ítem con `.findIndex()` (sin distinguir mayúsculas ni minúsculas) y reemplace su valor mediante `.splice()`. Muestra un mensaje adecuado si el ítem no fue encontrado.

► Punto 5: Eliminación de Ítems (Delete / pop, shift & splice)

Crea las siguientes funciones de baja de productos:

a) `eliminarUltimo()` con `.pop()`.

b) `eliminarPrimero()` con `.shift()`.

c) `eliminarPorNombre(nombreBuscado)` utilizando `.findIndex()` y `.splice()` para quitar exactamente 1 elemento.

► Punto 6: Buscador de Productos y Filtrado (find & filter)

Desarrolla las utilidades de consulta:

a) `buscarPrimerMatch(termino)` usando `.find()` para obtener la primera coincidencia que contenga la palabra buscada.

b) `filtrarPorCategoria(categoria)` usando `.filter()` para obtener un subconjunto de productos (por ejemplo: 'Café', 'Té', 'Tarta', 'Croissant', 'Muffin') ignorando diferencias de mayúsculas/minúsculas.

► Punto 7: Ordenamiento de la Carta (sort & reverse)

Implementa la función `ordenarCarta(modo)` para ordenar el array de menú:

a) En orden alfabético de A a Z mediante `.sort()`.

b) En orden inverso de Z a A combinando `.sort()` y `.reverse()`.

Muestra por consola la lista en cada ordenamiento.

► Punto 8: Resumen e Integración de Menús Especiales (slice, concat, reduce)

Resuelve las siguientes operaciones de unificación y copia:

a) Genera una copia de seguridad inmutable del array de menú con `.slice()`.

b) Define un nuevo array `menuTemporada = ["Smoothie Menta y Limón", "Frappé Caramelo Crocante", "Waffle con Dulce de Leche"]` y únelo al menú principal con `.concat()`.

c) Genera un resumen en un único string de texto continuo formateado que contenga todos los nombres del menú separados por comas o saltos de línea utilizando `.reduce()`.

► Punto 9: Sección de Investigación (some & every)

Investiga y aplica en código los métodos de orden superior `.some()` y `.every()`:
a) Verifica con `.some()` si en el menú existe al menos un producto que contenga la palabra 'Artesanal'.
b) Verifica con `.every()` si todos los nombres de productos del menú poseen más de 4 caracteres.

Operación CRUD	Métodos de Array a Utilizar	Resultado Esperado
Create (Crear)	push(), unshift(), splice()	Modificación directa del array insertando elementos en distintos puntos.
Read (Leer / Buscar)	forEach(), map(), find(), filter()	Recorrido numerado del menú, array de códigos, búsqueda única y por categoría.
Update (Actualizar)	findIndex(), splice()	Reemplazo de un nombre de producto en su posición exacta dentro del array.
Delete (Eliminar)	pop(), shift(), splice()	Eliminación del primer, último o de un ítem específico según término buscado.
Utilidades / Extras	sort(), reverse(), reduce(), slice(), concat(), some(), every()	Ordenamiento A-Z/Z-A, copias de respaldo, fusión de carta de temporada y validaciones.

4. Modalidad de Entrega y Repositorio de GitHub

El proyecto debe ser publicado de manera obligatoria en un repositorio público de GitHub. A continuación se especifican las pautas que deben cumplirse para la entrega:

- **Nombre del Repositorio:** `javascript-crud-cafeteria` o `typescript-crud-cafeteria`.
- **Estructura de Archivos:** El proyecto debe contar con un archivo de código fuente principal (`app.js`, `index.js` o `app.ts`) e incluir un archivo `README.md` explicativo.
- **Archivo README.md:** Debe incluir título del proyecto, descripción breve, instrucciones de ejecución (Node.js o navegador) y el listado de métodos de array empleados.
- **Historial de Commits:** Se evaluará un flujo de trabajo continuo mediante commits claros y significativos (mínimo 4 a 5 commits, ej: 'feat: add create operations', 'feat: implement category filter').
- **Entrega:** Subir el enlace directo del repositorio público de GitHub en la plataforma educativa.

5. Criterios de Evaluación

✓ Correcta resolución de todas las operaciones CRUD sobre el menú (30%).

✓ Uso preciso de métodos de orden superior (`forEach`, `map`, `find`, `filter`, `reduce`) mediante callbacks (25%).

✓ Cumplimiento estricto de las restricciones (no utilizar objetos ni estructuras de datos complejas) (15%).

✓ Investigación y uso adecuado de los métodos `.some()` y `.every()` (10%).

✓ Buenas prácticas de programación, legibilidad del código y nombres expresivos en variables/funciones (10%).

✓ Correcta estructuración, presentación y documentación del repositorio en GitHub con su README.md (10%).